



LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA
A.A. 2025/2026

Francesca Righetti
francesca.righetti@unipi.it



ESERCITAZIONE 5

Programmazione con i socket - I

Programma

- Introduzione al linguaggio C
- Programmazione con i socket

INTRODUZIONE AL LINGUAGGIO C, DIFFERENZE CON C++

Definizione di variabili

- Le variabili possono essere definite ovunque, sia per C++ che per C

Stile C++

```
int main() {  
    int a = 5, i, b;  
    a = func(1000);  
    int b = f(a);  
    // ...  
    for (i = 0; a < 100; ++i) {  
        b = f(a);  
        int c = 0;  
        // ...  
    }  
}
```

Stile C

```
int main() {  
    int a = 5, i, b;  
    a = func(1000);  
    int b = f(a);  
    // ...  
    for (i = 0; a < 100; ++i) {  
        b = f(a);  
        int c = 0;  
        // ...  
    }  
}
```

Strutture

- In C si deve specificare sempre **struct** quando si crea una variabile di tipo struttura:
 - Le struct sono solo contenitori di dati, non si possono definire funzioni membro al loro interno. Non sono previsti costruttori/distruttori automatici.

Stile C++

```
struct Complex {  
    double Re;  
    double Im;  
};  
  
int main() {  
    int a = 4;  
    Complex c;  
    // ...  
}
```

Stile C

```
struct Complex {  
    double Re;  
    double Im;  
};  
  
int main() {  
    int a = 4;  
    struct Complex c;  
    // ...  
}
```

Strutture

	C	C++
Dichiarazione	Scrivere sempre struct Nome per dichiarare variabili	Una volta definita, si può usare come tipo normale: Nome var;
Contenuto	Solo dati (campi)	Può contenere dati e metodi (funzioni membro)
Inizializzazione	Non ha costruttori automatici, va assegnato campo per campo	Può avere costruttori e distruttori.
Visibilità di default	Tutti i membri sono public	Anche qui tutti i membri sono public (ma in class sono private di default).
Ereditarietà	Non supportata	Supportata (una struct può ereditare da un'altra)
Operatore di overloading	Non disponibile	Disponibile (puoi ridefinire +, -, <<, ecc.)
Compatibilità	Le struct di C funzionano in C++.	Le struct di C++ possono avere funzionalità non compatibili con il C

Memoria dinamica

- Utile quando la dimensione di strutture dati non è nota a priori (es. array deciso dall'input utente)
- Allocata a runtime con funzioni della libreria <stdlib.h>
 - void *malloc(size_t size)
 - void free(void *ptr)
- Rimane allocata finché non liberata esplicitamente con free()

```
#include <stdlib.h>

int main() {
    int mem_size = 5; //byte
    void *ptr;
    ptr = malloc(mem_size);
    if (ptr == NULL) {
        // Gestione errore
    }
    // ...
    free(ptr);
}
```

```
#include <stdlib.h>

int main() {
    int mem_size = 5; //byte
    int *ptr;
    ptr = (int*)malloc(sizeof(int)*mem_size);
    // 4bytes * 5 = 20 bytes
    if (ptr == NULL) {
        // Gestione errore
    }
    // ...
    free(ptr);
}
```


Operazioni di I/O

- Operazioni di I/O si basano sulla libreria standard <stdio.h>
 - fornisce funzioni per leggere e scrivere da/verso
 - stdin (tastiera) -> int scanf(char* formato, indirizzi)
 - scanf è rischiosa con stringhe, perché non controlla la lunghezza e può causare buffer overflow,
 - Meglio fgets(str, sizeof(str), stdin)
 - stdout (schermo) -> int printf(char* formato, argomenti)
 - File -> fprintf(FILE *f, ...), fscanf(FILE *f, ...)

```
#include <stdio.h>

char *str = "Hello!\n";
printf(str);
printf("str = %s", str);

printf("Hello World!\n");

int i = 5;
printf("i = %d\n", i);

scanf("%d", &i);
printf("i = %d\n", i);
```

%d, %i	intero con segno (decimale)
%u	intero senza segno (decimale)
%o	intero in ottale
%x, %X	intero in esadecimale
%f, %F	floating point decimale
%e, %E	notazione scientifica
%g, %G	formato compatto (tra decimale e scientifico)
%a, %A	floating point in notazione esadecimale
%c	carattere singolo
%s	stringa (terminata da \0)
%p	puntatore / indirizzo di memoria
%n	numero di caratteri già scritti
%%	carattere % letterale

Stringhe

- In C non esiste tipo nativo stringa, una stringa è un **array di caratteri (char) terminato** dal carattere speciale **\0**

```
#include <string.h>
```

```
// Lunghezza
```

```
char *str1 = "Hello \n";
```

```
int len;
```

```
len = strlen(str1);
```

```
// Confronto
```

```
char *str2 = "World!\n";
```

```
int ret;
```

```
ret = strcmp(str1, str2);
```

H	e	l	l	o		\n	\0
---	---	---	---	---	--	----	----

8 byte allocati, ma `strlen()` restituisce 7!

ret = 0 se le stringhe sono identiche

ret < 0 se **str1** è alfabeticamente minore di **str2**

ret > 0 se **str1** è alfabeticamente maggiore di **str2**

Stringhe

```
#include <string.h>

// Copia
char str[20];
n = sizeof(str);
strncpy(str, "Hello \n", n);

// Concatenazione
char *str2 = "World!\n";
n2 = sizeof(str2);
strcat(str, str2, n2);
```

- Fare attenzione ad aggiungere in size anche lo spazio per copiare il terminatore di stringa
- Le funzioni non controllano che ci sia spazio nella stringa di destinazione

H	e	l	l	o		\n	W	o	r	l	d	!	\n	\0
---	---	---	---	---	--	----	---	---	---	---	---	---	----	----

Gestione dei file

- Ogni file aperto è rappresentato da un puntatore di tipo FILE *, definito nella libreria <stdio.h>

```
#include <stdio.h>

// Apertura file
FILE *fd;
fd = fopen("/tmp/foo.txt", "r" );
if (fd == NULL) {
    // Gestione errore
}
```

Si specifica un percorso (assoluto o relativo) e una modalità di apertura:

- **r** = sola lettura
- **w** = sola scrittura
- **r+** = lettura e scrittura
- **a** = *append*
- **a+** = lettura e *append*

Se il file non esiste e si specifica scrittura o *append*, il file viene creato

Gestione dei file

```
#include <stdio.h>

// Lettura file
int ret, n;
FILE *fd1;
fd1 = fopen("/tmp/foo.txt", "r");
if (fd1 == NULL) {
    // Gestione errore
}
ret = fscanf(fd1, "%d", &n);

// Scrittura file
char *str = "Hello!\n";
FILE *fd2;
fd2 = fopen("/tmp/bar.txt", "w");
if (fd2 == NULL) {
    // Gestione errore
}
ret = fprintf(fd2, "%s", str);
```

Si comportano allo stesso modo di **scanf()** e **printf()**, ma usano il file specificato invece di **stdin** e **stdout**

Gestione dei file

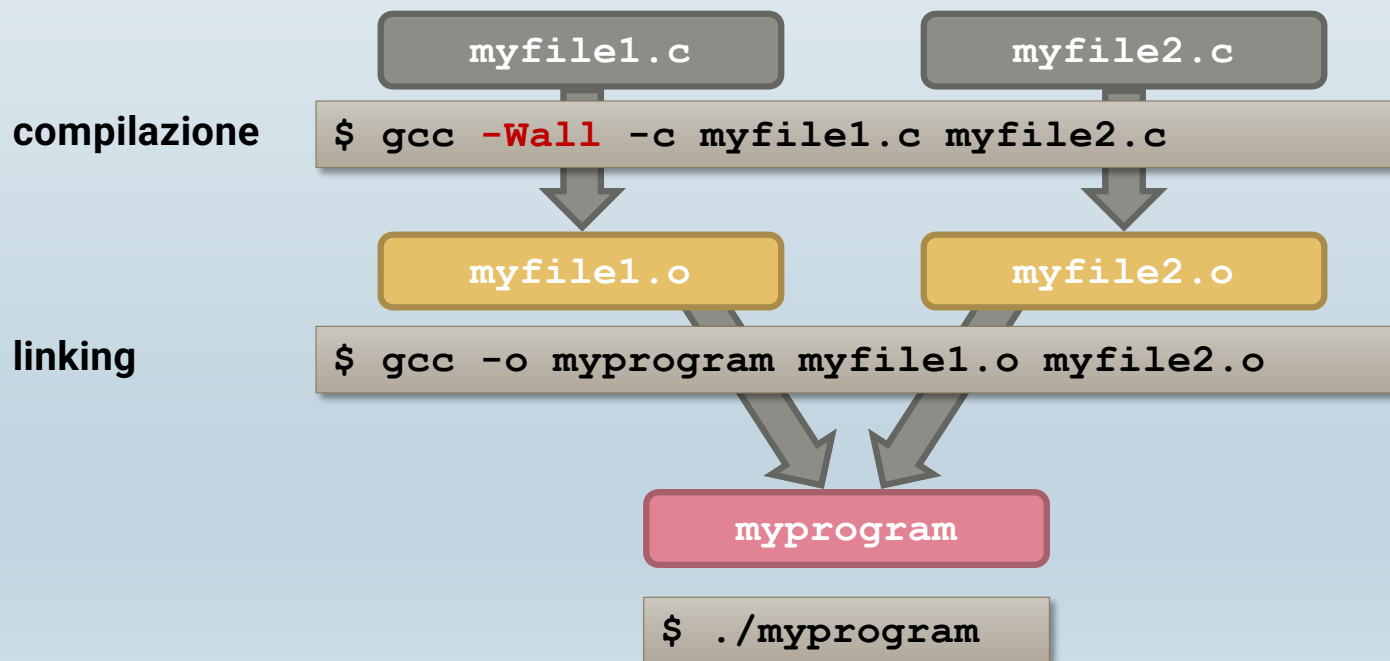
```
#include <stdio.h>
#include <sys/stat.h>

// Dimensione
int ret, size;
struct stat info;
ret = stat("/tmp/foo.txt", &info);
size = info.st_size;
```

Non serve aprire il file per usare **stat()**

Compilazione

- Useremo la *GNU Compiler Collection* (GCC) (man gcc)
 - Con la **compilazione**, si creano i file oggetto a partire dai file sorgente
 - Con il **linking**, si crea un file eseguibile a partire dai file oggetto



PROGRAMMAZIONE DISTRIBUITA

CONCETTI GENERALI

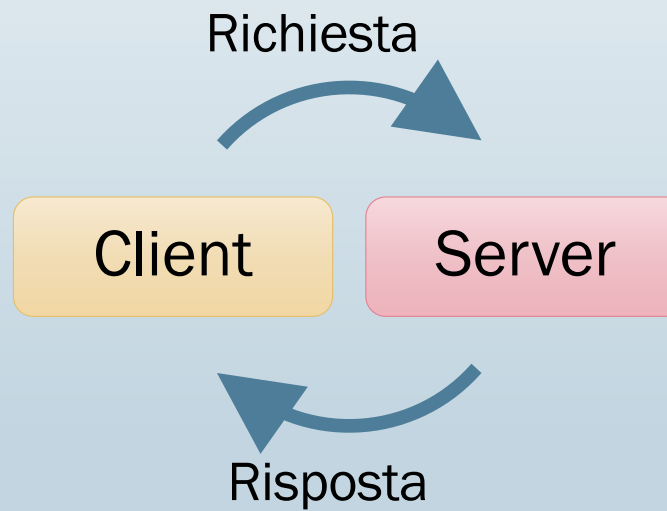
Cooperazione tra processi

- Due processi possono cooperare attraverso:
 - Sincronizzazione (Es. semafori)
 - Comunicazione, cioè scambio di informazioni
 - Memoria condivisa
 - Chiamate a procedura remota
 - Scambio di messaggi

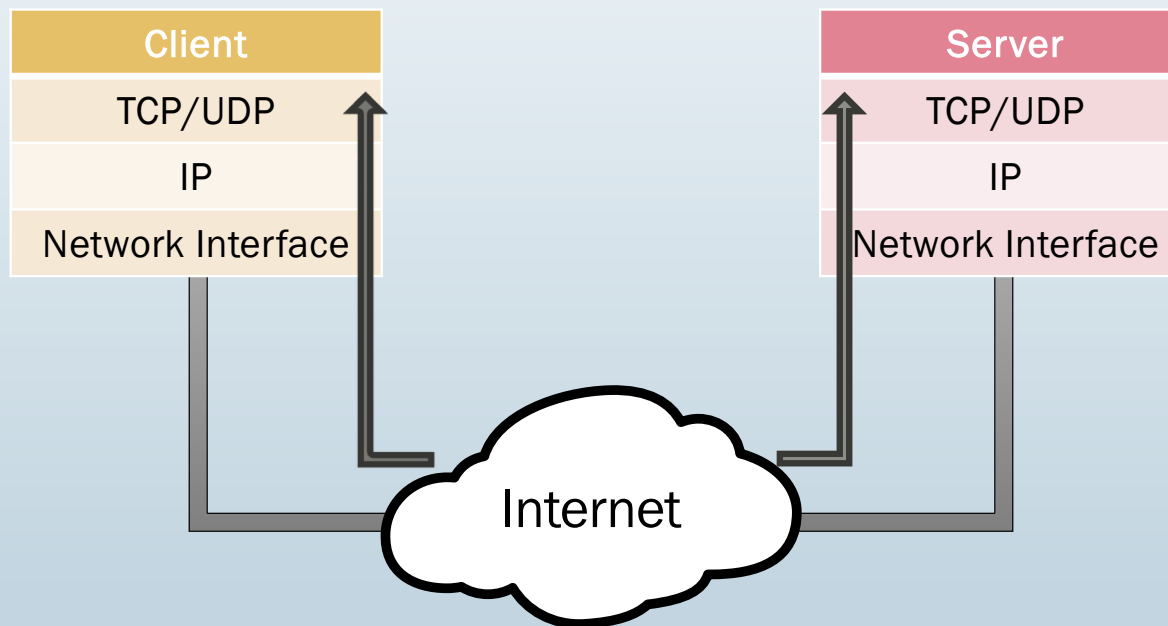
- Due processi possono cooperare
 - Sulla stessa macchina
 - **Su macchine diverse (sistema distribuito)**
 - Come avviene la comunicazione ? Tramite scambio di messaggi per cui è stata definita l'astrazione dei **socket**

Client/Server

- Paradigma basato su scambio di messaggi
- Viene usato principalmente per sistemi distribuiti



Client/Server

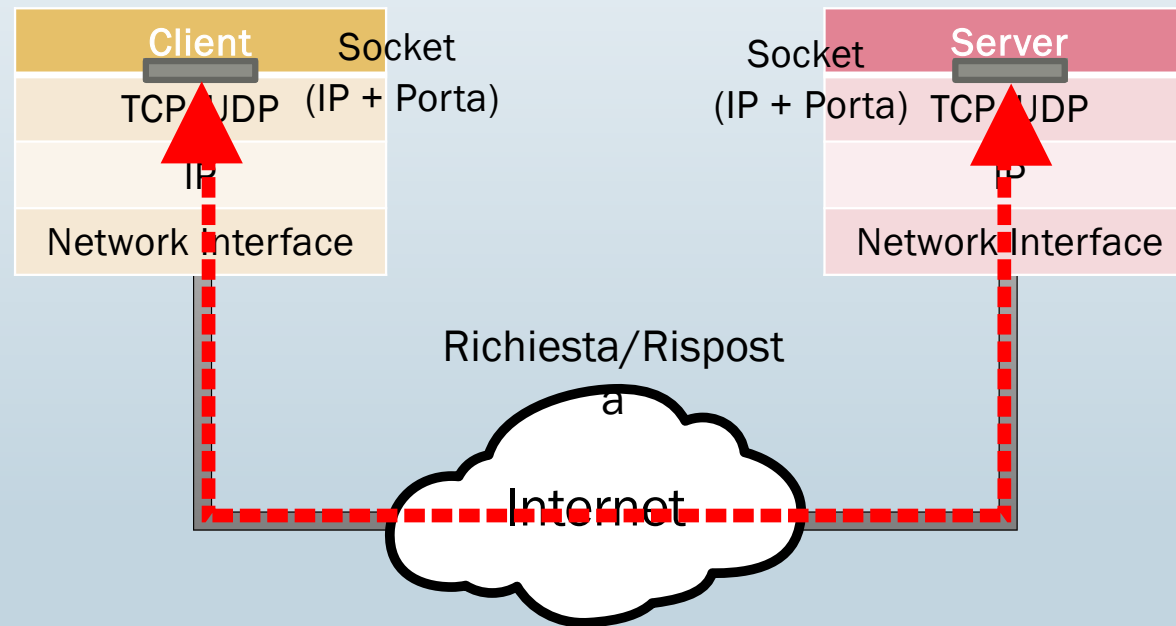


Socket

- Meccanismo per la comunicazione tra processi
 - Sulla stessa macchina o su macchine differenti
- È un'astrazione:
 - Un' interfaccia unica per operare con diversi protocolli di rete
 - Nasconde i dettagli dei livelli sottostanti
- Un socket è identificato da un indirizzo:
 - Indirizzo host (TCP/IP: Indirizzo IP)
 - Indirizzo processo (TCP/IP: Numero di porta)

Socket

- Un socket è l'estremità di un canale di comunicazione



Socket

- L'astrazione del socket è implementata dal SO
- Abbiamo a disposizione delle **primitive** (system calls) per:
 - Creare un socket
 - Assegnargli un indirizzo e una porta
 - Connettersi ad un altro socket
 - Accettare una connessione
 - Inviare e ricevere dati attraverso i socket
 - ...

Socket – primitiva socket ()

- Crea un socket

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int socket(int domain, int type, int protocol);
```

- **domain:** famiglia di protocolli da utilizzare
 - **AF_LOCAL** – Comunicazione locale
 - **AF_INET** – Protocolli IPv4, TCP e UDP
- **type:** tipologia di socket
 - **SOCK_STREAM** – Connessione affidabile, bidirezionale (TCP) - Oggi vedremo questo
 - **SOCK_DGRAM** – Invio di pacchetti senza connessione (UDP)
- **protocol:** sempre a 0 (potrebbe specificare un altro protocollo di trasporto)
- La funzione restituisce
 - un descrittore di file che rappresenta il socket e servirà a manipolare il socket attraverso altre primitive
 - -1 su errore
- **Attenzione:** il socket non è ancora associato a un indirizzo IP o a una porta

Socket – Strutture per gli indirizzi

```
/* Endpoint IPv4 in netinet/in.h */

struct sockaddr_in {
    sa_family_t    sin_family; /* address family: AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr sin_addr;   /* internet address */
};

/* Internet address in netinet/in.h */
struct in_addr {
    uint32_t       s_addr;     /* address in network byte order */
};
```

- **Attenzione:** esiste anche la struttura **struct sockaddr** usata da alcune funzioni descritte più avanti ma non la useremo direttamente.

Socket – Ordine dei byte - Endianess

- Calcolatori diversi possono usare modalità diverse per **ordinare i byte**
 - Esempio: numero **422990**, in 32 bit (4 byte)
 - 0000 0000 0000 0110 0111 0100 0100 1110
 - **Memorizzazione big-endian** - Per primo il byte più significativo (MSB)

Indirizzo A	Indirizzo A+1	Indirizzo A+2	Indirizzo A+3
0000 0000	0000 0110	0111 0100	0100 1110

- **Memorizzazione little-endian** - Per primo il byte meno significativo (LSB)

Indirizzo A	Indirizzo A+1	Indirizzo A+2	Indirizzo A+3
0100 1110	0111 0100	0000 0110	0000 0000

Socket – Ordine dei byte

- Il formato di rete (*network order*), usato nei socket, è *big-endian*
- Il formato dell'host (*host order*) dipende dal singolo host
- Funzioni di conversione:

```
#include <arpa/inet.h>
```

```
uint32_t htonl(uint32_t hostlong); // host to network long  
uint16_t htons(uint16_t hostshort); // host to network short  
uint32_t ntohl(uint32_t netlong); // network to host long  
uint16_t ntohs(uint16_t netshort); // network to host short
```

- Attenzione: uint16_t e uint32_t sono interi senza segno che occupano sempre 16 e 32 bit a prescindere dal calcolatore usato per compilare. Sono utili quando si vuole avere il controllo completo della dimensione delle variabili, come in questo caso.
- Sono definiti nell'header <stdint.h>

Socket – Formato degli indirizzi

- Formato **numerico (n)**: 32-bit, usato dal computer (3223455 099)
- Formato **presentazione (p)**: stringa in notazione decimale puntata (192.34.5.123)

```
int inet_pton(int af, const char *src, void *dst);
```

- **af**: (address family) famiglia (AF_INET)
- **src**: stringa del tipo "ddd.ddd.ddd.ddd"
- **dst**: puntatore a una struct in_addr

```
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
```

- **af**: famiglia (AF_INET)
- **src**: puntatore a una struct in_addr
- **dst**: puntatore a un buffer di caratteri lungo size
- **size**: deve valere almeno INET_ADDRSTRLEN

Socket – Iniziamo a fare qualcosa

```
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

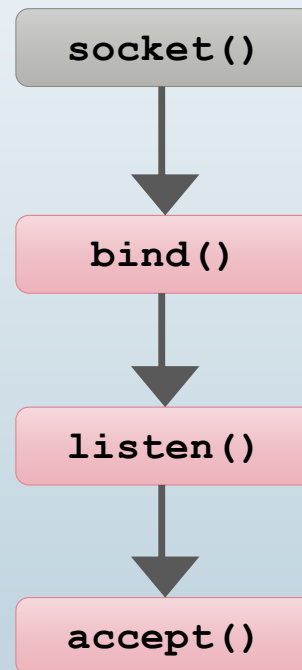
int main () {
    /* Creazione socket */
    int sd = socket(AF_INET, SOCK_STREAM, 0);

    /* Creazione indirizzo */
    struct sockaddr_in my_addr;
    memset(&my_addr, 0, sizeof(my_addr)); // Pulizia
    my_addr.sin_family = AF_INET ;
    my_addr.sin_port = htons(4242);
    inet_pton(AF_INET, "192.168.4.5", &my_addr.sin_addr);
```

PROGRAMMAZIONE DISTRIBUITA

LATO SERVER

Primitive per la creazione socket - SERVER



Socket – Primitiva bind()

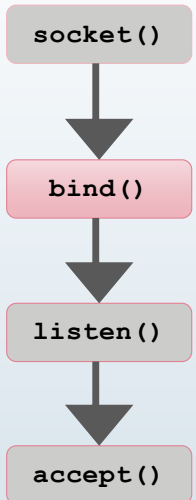
- **Assegna** un socket a un indirizzo (IP+Porta)

- Viene usata dal server per specificare indirizzo e porta sui quali ricevere richieste
 - Di solito, il client non ha bisogno di eseguire la **bind()**

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- **sockfd**: descrittore del socket (socket file descriptor)
- **addr**: puntatore alla struttura di tipo struct sockaddr
 - Visto che usiamo struct sockaddr_in bisogna convertire il puntatore
- **addrlen**: dimensione di addr
- La funzione restituisce 0 se ha successo, -1 su errore

```
ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));
```



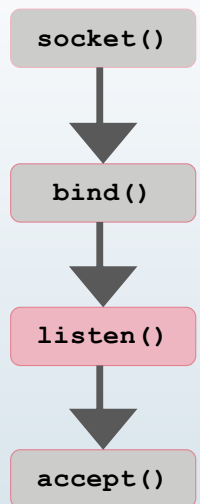
Socket – Primitiva listen()

- Specifica che il socket è **passivo**, cioè verrà usato per ricevere richieste di connessione
 - Come vedremo, si possono mettere in attesa solo i socket **SOCK_STREAM**

```
int listen(int sockfd, int backlog);
```

- **sockfd**: descrittore del socket
- **backlog**: dimensione della coda, cioè quante richieste dai client possono rimanere in attesa di essere gestite
- La funzione restituisce 0 se ha successo, -1 su errore

```
ret = listen(sd, 10);
```



La richiesta (da parte di un client) di connessione al socket del server arriva al sistema operativo:

- Il kernel la mette in **attesa** in una coda, finché il server non chiama accept() per prenderla in carico.
- Se arrivano più richieste contemporanee, vengono accodate.

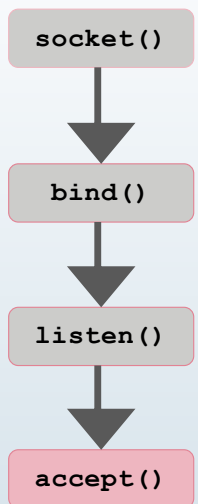
Socket – Primitiva accept()

- **Accetta una** richiesta di connessione pervenuta sul socket
 - Anche in questo caso, ha senso solo sui socket **SOCK_STREAM**

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- **sockfd**: descrittore del socket
- **addr**: puntatore a una struttura (vuota) di tipo **struct sockaddr**
 - Qui ci viene salvato l'indirizzo del client
- **addrlen**: dimensione di **addr**
- La funzione restituisce il **descrittore di un nuovo socket** che verrà usato per la comunicazione, -1 su errore
- **La funzione è bloccante**: il programma si ferma finché non arriva una richiesta

```
struct sockaddr_in cl_addr;  
int len = sizeof(cl_addr);  
new_sd = accept(sd, (struct sockaddr*)&cl_addr, &len);
```



Socket – Codice del server

```
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main () {
    int ret, sd, new_sd, len;
    struct sockaddr_in my_addr, cl_addr; // Due strutture!
    /* Creazione socket */
    sd = socket(AF_INET, SOCK_STREAM, 0);
    /* Creazione indirizzo */
    memset(&my_addr, 0, sizeof(my_addr)); // Pulizia
    my_addr.sin_family = AF_INET ;
    my_addr.sin_port = htons(4242);
    inet_pton(AF_INET, "192.168.4.5", &my_addr.sin_addr);
    // In alternativa socket in ascolto su tutte le interfacce:
    my_addr.sin_addr.s_addr = INADDR_ANY;
    ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));
    ret = listen(sd, 10);
    int len = sizeof(cl_addr);
    new_sd = accept(sd, (struct sockaddr*)&cl_addr, &len);
    //...
}
```

PROGRAMMAZIONE DISTRIBUITA

LATO CLIENT

Socket – Primitiva connect()

- **Connette** il socket locale ad un socket remoto

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- **sockfd**: descrittore del socket (locale)
- **addr**: puntatore alla struttura contenente l'indirizzo del server
- **addrlen**: dimensione di addr
- La funzione restituisce 0 se ha successo, -1 su errore
- **La funzione è bloccante**: il programma si ferma finché la richiesta di connessione non è stata accettata

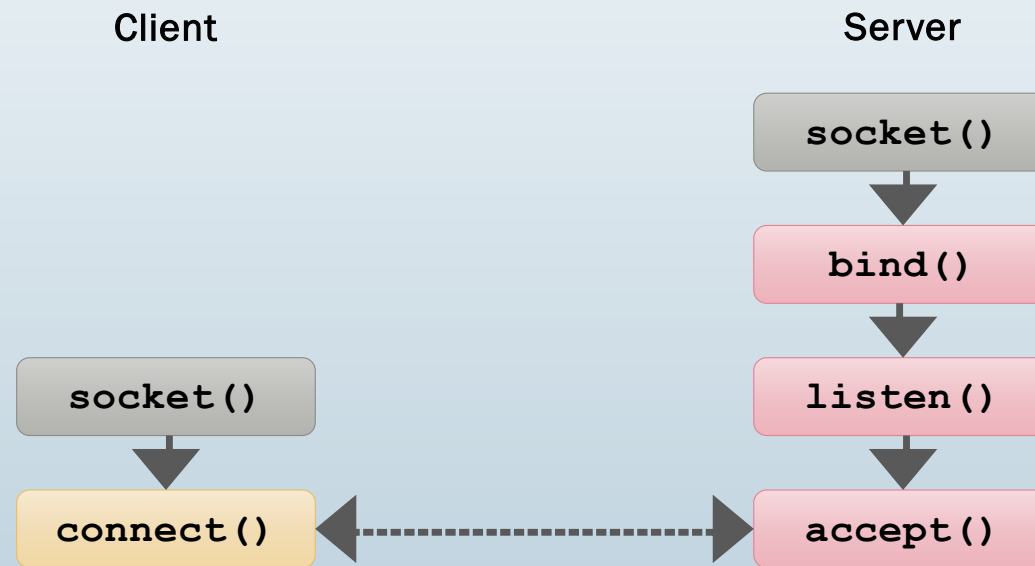
```
ret = connect(sd, (struct sockaddr*)&sv_addr, sizeof(sv_addr));
```

Socket – Codice del client

```
#include <arpa/inet.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main () {
    int ret, sd;
    struct sockaddr_in sv_addr; // Struttura per il server
    /* Creazione socket */
    sd = socket(AF_INET, SOCK_STREAM, 0);
    /* Creazione indirizzo del server */
    memset(&sv_addr, 0, sizeof(sv_addr)); // Pulizia
    sv_addr.sin_family = AF_INET ;
    sv_addr.sin_port = htons(4242);
    inet_pton(AF_INET, "192.168.4.5", &sv_addr.sin_addr);
    ret = connect(sd, (struct sockaddr*)&sv_addr,
        sizeof(sv_addr));
    //...
}
```

Socket – Recap, cosa succede?



PROGRAMMAZIONE DISTRIBUITA

SCAMBIO DATI

Socket – Primitiva send()

- Invia un messaggio attraverso un socket **connesso**

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

- **sockfd**: descrittore del socket
- **buf**: puntatore al buffer contenente il messaggio da inviare
- **len**: dimensione in byte del messaggio
- **flags**: per settare delle opzioni, lasciamolo a 0
- La funzione restituisce il numero di byte inviati, -1 su errore
- **La funzione è bloccante**: il programma si ferma finché non ha scritto tutto il messaggio
 - send() non “spedisce pacchetti”, ma copia i dati nel buffer del kernel, che poi li frammenterà in pacchetti TCP.
 - Se il buffer del kernel è quasi pieno (perché la rete è lenta o il destinatario non legge abbastanza veloce), send():
 - blocca l’esecuzione finché c’è spazio (perché è bloccante)

Socket – Primitiva send()

```
int ret, sd, len;  
char buffer[1024];  
  
//...  
  
strcpy(buffer, "Hello Server!");  
len = strlen(buffer);  
ret = send(sd, (void*)buffer, len, 0);  
if (ret < len) {  
    // Gestione errore  
}
```

Socket – Primitiva recv()

- Preleva un messaggio da un socket **connesso**

```
ssize_t recv(int sockfd, const void *buf, size_t len, int flags);
```

- **sockfd**: descrittore del socket
- **buf**: puntatore al buffer in cui salvare il messaggio
- **len**: dimensione in byte del messaggio desiderato
- **flags**: per settare delle opzioni
- La funzione restituisce il numero di byte ricevuti, -1 su errore, 0 se il socket remoto si è chiuso (vedi più avanti)
- La **funzione è bloccante**: il programma si ferma finché non ha letto qualcosa

Socket – Primitiva recv()

```
int ret, sd, bytes_needed;  
char buffer[1024];  
  
//...  
  
bytes_needed = 20;  
ret = recv(sd, (void*)buffer, bytes_needed, 0);  
// Adesso 0 < ret <= bytes_needed  
if (ret < bytes_needed) {  
    // Gestione errore  
}  
  
ret = recv(sd, (void*)buffer, bytes_needed, MSG_WAITALL);  
// Adesso ret == bytes_needed
```

Socket – Primitiva close()

- **Chiude** un socket

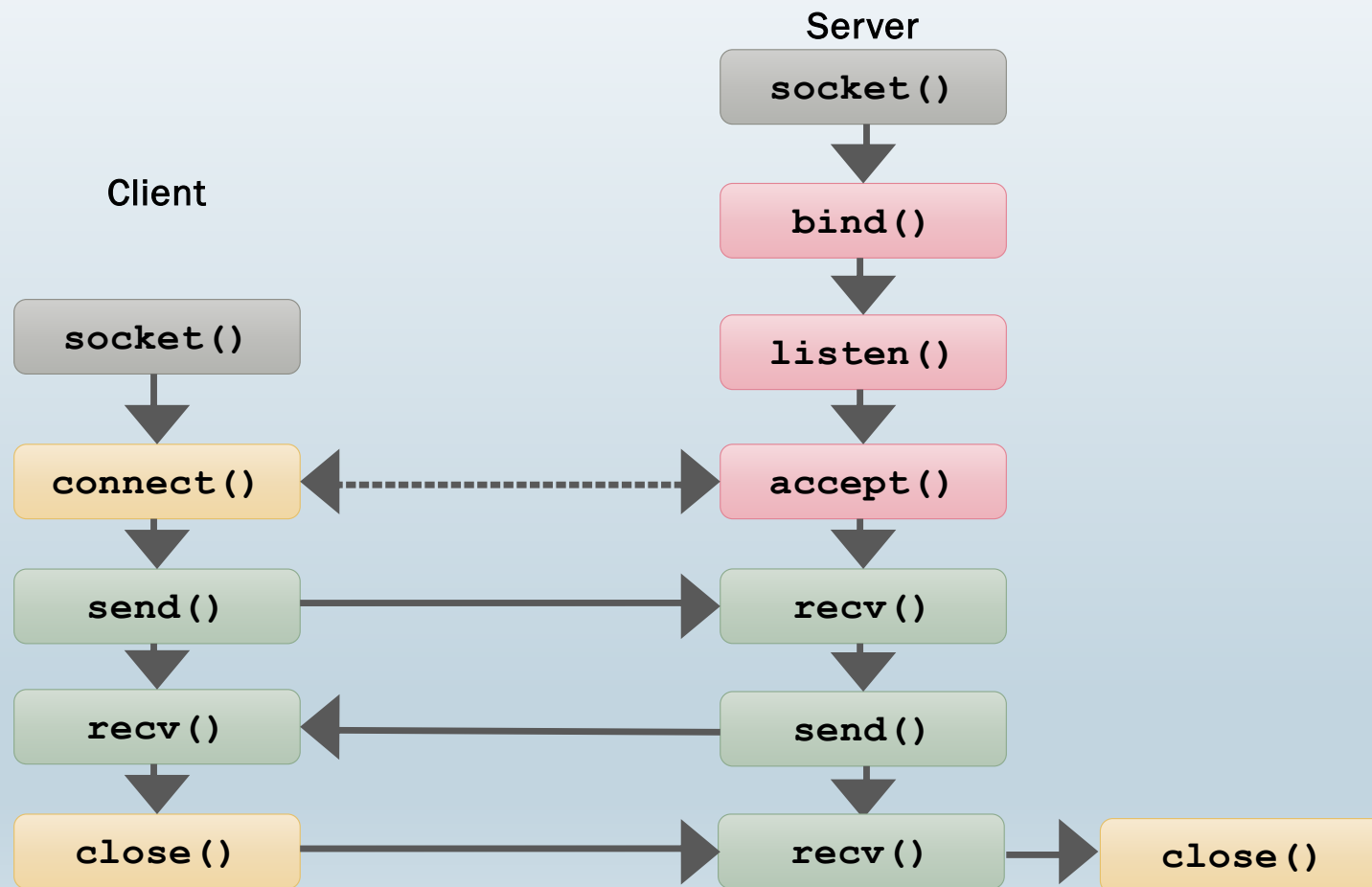
- Non può più essere usato per inviare o ricevere dati

```
#include <unistd.h>
```

```
int close(int fd);
```

- **fd**: descrittore del socket
- La funzione restituisce 0 se ha successo, -1 su errore
- L'host remoto riceverà 0 dalla recv()

Socket – Recap, cosa succede?



GESTIONE DEGLI ERRORI

Gestione degli errori

- Le primitive viste finora restituiscono -1 quando c'è un errore
- In più settano una variabile, errno, che può essere letta per scoprire il motivo dell'errore
- Nel manuale di ogni funzione c'è l'elenco degli errori possibili

```
#include <errno.h>
//...
ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));
if (ret == -1) {
    if (errno == EADDRINUSE) { /* Gestisci errore */}
    if (errno == EINVAL) { /* Gestisci errore */}
    //...
}
```

Gestione degli errori

Valore	Nome costante	Significato	Quando capita tipicamente
9	EBADF	Bad file descriptor	Il socket non è valido o è stato già chiuso.
13	EACCES	Permission denied	Porta privilegiata (<1024) senza privilegi root.
98	EADDRINUSE	Address already in use	<code>bind()</code> su una porta già occupata.
99	EADDRNOTAVAIL	Cannot assign requested address	IP non valido o non configurato sull'host.
111	ECONNREFUSED	Connection refused	Il server non è in ascolto sulla porta indicata.
113	EHOSTUNREACH	No route to host	Nessuna rotta verso l'host remoto.
110	ETIMEDOUT	Connection timed out	Nessuna risposta dal server entro il timeout.
32	EPIPE	Broken pipe	Scrittura (<code>send</code>) su un socket chiuso dall'altro lato.
104	ECONNRESET	Connection reset by peer	L'altro lato ha chiuso in modo "brusco" la connessione.
105	ENOBUFS	No buffer space available	Buffer di rete saturi, sistema sotto pressione.
115	EINPROGRESS	Operation now in progress	In socket non-bloccanti durante <code>connect()</code> .
11	EAGAIN/EWOULDBLOCK	Resource temporarily unavailable	Nessun dato pronto in socket non-bloccante (o coda piena in <code>send()</code>).
4	EINTR	Interrupted system call	La <code>recv()</code> / <code>send()</code> è stata interrotta da un segnale.

Gestione degli errori

- A volte vogliamo solo sapere l'errore e uscire
 - **perror()** legge **errno** e stampa l'errore su schermo in forma leggibile

```
#include <stdio.h>
//...
ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));
if (ret == -1) {
    perror("Error: ");
    exit(1);
}
//...
```